Cloud PKI Integration with Intune (Certificate Management Modernization)

Implementation Solution Guide

Version 1.0.0 · 2026-06-30

Contents

Executive Summary — Cloud PKI Integration with Intune (Certificate Management Modernization) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Cloud PKI Integration with Intune (Certificate Management Modernization) 5

What a cloud PKI integration with Intune is . . . 5

Why modernize off an on-prem CA (the *why*) . . . 5

How it works . . . 5

SCEP vs PKCS (which profile to use) . . . 6

What you can control . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Cloud PKI Integration with Intune 7

2.1 Licensing . . . 7

2.2 Device enrollment . . . 7

2.3 Cloud PKI Intune connector / plugin connected . . . 7

2.4 Trust chain available (root + any intermediate) . . . 7

2.5 Issuing CA / certificate template ready in the cloud PKI . . . 7

2.6 Certificate scope groups (pilot + broad) . . . 8

2.7 Certificate-backed scenario targets (Wi-Fi / VPN / auth) . . . 8

2.8 Legacy CA inventory (for migration) . . . 8

2.9 Change control / rollback (authentication-impacting) . . . 8

Checklist . . . 8

3. Implementation Guide — Cloud PKI Integration with Intune 9

Step 1 — Create the scope groups . . . 9

Step 2 — Connect the cloud PKI to Intune (vendor console) . . . 9

Step 3 — Deploy the trusted root/intermediate chain (do this before any issuance) . . . 10

Step 4 — Create the SCEP device certificate profile (pilot) . . . 10

Step 5 — (If issuing user certificates) Create the PKCS user certificate profile (pilot) . . . 11

Step 6 — Point a certificate-backed scenario at the issued certificate (pilot) . . . 11

Step 7 — Apply on a pilot device and validate . . . 12

Step 8 — Pilot gate . . . 12

Step 9 — Migrate workloads off the legacy CA (coexistence → retire) . . . 12
Step 10 — Expand to production . . . 13
5. Testing Guide — Cloud PKI Integration with Intune 13
Test 1 — Connector healthy . . . 13
Test 2 — Trust chain installed on the device . . . 13
Test 3 — Certificate issued to the device . . . 13
Test 4 — Certificate properties correct . . . 13
Test 5 — Wi-Fi scenario works on the cloud certificate . . . 13
Test 6 — VPN / auth scenario works on the cloud certificate . . . 14
Test 7 — Renewal behaves . . . 14
Test 8 — Coexistence (during migration) . . . 14
Test 9 — Fleet coverage + expiry report . . . 14
Pass/fail gate . . . 14
6. Troubleshooting Guide — Cloud PKI Integration with Intune 14
A. Connector unhealthy / no issuance . . . 14
B. Trust chain problems . . . 15
C. Certificate not issuing to the device . . . 15
D. Wi-Fi / VPN / auth scenario fails on the new certificate . . . 15
E. Migration / coexistence issues . . . 16
F. Expiry / renewal . . . 16
G. Diagnostics quick reference . . . 16
H. When to escalate . . . 16
7. Best Practices — Cloud PKI Integration with Intune 16
8. Deliverables Summary — Cloud PKI Integration with Intune 17
Documentation . . . 17
PowerShell scripts . . . 18
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Cloud PKI Integration with Intune (Certificate Management Modernization)

Certificates are everywhere in a modern fleet — Wi-Fi, VPN, and user/device authentication all depend on them — yet many organizations still issue them from an aging on-prem certification authority (CA) chained to Intune through a fragile connector. That model is hard to scale, hard to make highly available, and a single point of failure for network access. A **cloud PKI with an Intune integration** modernizes this: certificates are issued to Intune-managed devices through the cloud PKI's Intune connector/plugin, the trust chain is deployed by Intune, and the on-prem CA dependency is reduced or retired.

This is a complete, deploy-it-today package for integrating a cloud PKI with Microsoft Intune, written for an administrator with no prior PKI-modernization experience. It is **vendor-neutral** — the specific cloud-PKI product's console and connector internals are deferred to that vendor's documentation, while this solution gives the correct, repeatable Intune-side procedure that applies to any cloud PKI with an Intune integration.

What you get

- A plain-language **Solution Overview** of how a cloud PKI integrates with Intune (connector/plugin, trust chain, SCEP vs PKCS issuance) and what migrating off an on-prem CA looks like.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** of precise, ordered Intune steps: connect the cloud PKI, deploy the trusted root/intermediate chain, create SCEP/PKCS certificate profiles, issue to a pilot, validate certificate-backed scenarios, then migrate workloads off the legacy CA.
- Five **PowerShell scripts** for connector/profile readiness discovery, per-device issuance validation, certificate coverage + expiry reporting, safe remediation of failed issuance, and certificate-scope group building.
- **Testing** and **Troubleshooting** guides with explicit pass/fail gates.

What this delivers

A working **cloud PKI** ↔ **Intune integration**: the connector connected, the trusted root/intermediate chain deployed, SCEP and/or PKCS certificate profiles issuing certificates to a validated pilot, the certificate-backed scenarios (Wi-Fi, VPN, authentication) confirmed, and certificate workloads migrated off the legacy on-prem CA through a coexistence-then-retire path.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand**, with a mandatory pilot gate and a documented rollback before any production assignment.

Outcome

Certificates issued to Intune-managed devices from a cloud PKI — trust chain deployed, Wi-Fi/VPN/auth validated, and the on-prem CA dependency reduced or retired — proven on a pilot before you scale.

1. Solution Overview — Cloud PKI Integration with Intune (Certificate Management Modernization)

What a cloud PKI integration with Intune is

A **cloud PKI** is a hosted certification authority (CA) service that issues and manages the certificates your fleet needs — for Wi-Fi, VPN, and user/device authentication — without you running and maintaining an on-prem CA. When that cloud PKI offers an **Intune integration** (a connector or plugin registered to your tenant), Intune becomes the delivery mechanism: it deploys the cloud PKI's **trust chain** to devices and issues certificates from the cloud PKI through standard Intune **SCEP** and **PKCS** certificate profiles. The result is the same certificate-backed scenarios you have today, but issued from the cloud and managed end-to-end through Intune.

This solution is **vendor-neutral**: it treats the cloud PKI as "a cloud PKI with an Intune connector/plugin." The specific product's console, connector installation, and CA-template internals are deferred to that vendor's documentation. What you get here is the correct, repeatable **Intune-side** procedure that works regardless of which cloud PKI you choose.

Why modernize off an on-prem CA (the *why*)

- **Fragility.** An on-prem CA plus the legacy Intune **certificate connector (NDES/SCEP)** is a multi-server chain (CA, NDES, connector, IIS) that is hard to make highly available and is a single point of failure for network access.
- **Scale and maintenance.** Patching, certificate-template management, and connector upkeep are ongoing burdens; the cloud PKI shifts CA operations to the vendor.
- **Cert sprawl and visibility.** Without central reporting it is hard to know which devices have valid certificates and which are about to expire. A cloud PKI plus Intune profiles gives one consistent issuance and reporting surface.
- **Reduced on-prem footprint.** Retiring the on-prem CA dependency removes servers, firewall exceptions, and a class of outages from the environment.

How it works

1. The cloud PKI's **Intune connector/plugin** is connected to your tenant (per the vendor's documentation), so Intune can request certificates from the cloud PKI's issuing CA.
1. Intune deploys the cloud PKI's **trusted root** and any **intermediate** certificate(s) to devices via **Trusted certificate** profiles — the chain must be present before any leaf certificate is useful.
1. Intune **SCEP** certificate profiles (challenge-based, the private key generated on-device) and/or **PKCS** certificate profiles (CA-issued through the connector) request certificates from the cloud PKI and install them on assigned devices.
1. Certificate-backed profiles — **Wi-Fi**, **VPN**, and authentication scenarios — reference the

issued certificate, so devices authenticate using the cloud-issued identity.

1. Issuance and trust status is observable in Intune (per-profile device status) and via the included reporting scripts (coverage + expiry).

SCEP vs PKCS (which profile to use)

• **SCEP** — the device generates its own private key and requests a certificate using a challenge. Best for **device** certificates and the broadest platform support; the private key never leaves the device.

• **PKCS** — the certificate is issued through the connector and delivered to the device. Common for **user** certificates and scenarios that need a specific subject/SAN from directory attributes.

• Many fleets use **both**: SCEP for device authentication (Wi-Fi/VPN), PKCS for user authentication. Confirm which your cloud PKI's Intune integration supports — most support SCEP, many support PKCS.

What you can control

• Which **trust chain** (root + intermediate) is deployed, and to whom.

• The **subject name** and **SAN** format of issued certificates (from device/user attributes).

• **Key size/usage**, **validity period**, and **renewal threshold** (auto-renew before expiry).

• The **scope** of issuance via Entra security groups (pilot first, then broad).

• The certificate-backed **Wi-Fi/VPN/auth** profiles that consume the issued certificate.

Limitations (be honest with buyers)

• The **cloud PKI product itself is out of scope** here — its console, connector install, and CA configuration follow the **vendor's** documentation. This solution covers the Intune integration and rollout.

• **PKCS** support and exact SCEP challenge/connector behavior **vary by cloud PKI**; confirm against the vendor's docs before designing user-certificate flows.

• **Migration off a legacy CA is a coexistence exercise**, not a switch flip — old and new chains must both be trusted during the transition, and certificate-backed profiles (Wi-Fi/VPN) must be re-pointed carefully to avoid lockouts.

• Some scripts that touch **cloud-PKI-product-specific** APIs are flagged and deferred to vendor docs; the Intune/Graph-side discovery, validation, and reporting are fully covered.

The standard deployment model

> ***Connect the cloud PKI → deploy the trusted chain → create SCEP/PKCS profiles → issue to a pilot → validate Wi-Fi/VPN/auth → migrate workloads off the legacy CA (coexist, then retire) → expand.*** *Pattern:* ***Assess → Configure → Pilot → Validate → Expand****, with a pilot gate and rollback.*

2. Requirements Guide — Cloud PKI Integration with Intune

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-security.md`.

> ***Vendor note:*** *the cloud-PKI product's own subscription, console, connector install, and CA configuration are deferred to* ***the vendor's documentation****. The requirements below cover what must be true on the* ***Intune/Entra*** *side and the integration touch-points.*

2.1 Licensing

Description: Intune (Plan 1 or via M365 E3/E5 / Business Premium) plus a **cloud PKI subscription with an Intune integration**. **Why:** Intune delivers the certificate profiles and trust chain; the cloud PKI is the issuing CA. **Verify:** M365 admin center → **Billing** → **Licenses** (Intune); the cloud PKI vendor's portal shows an active subscription with the Intune connector/plugin feature. **Implement:** Assign Intune licensing; procure the cloud PKI subscription tier that includes its Intune integration (per vendor docs).

2.2 Device enrollment

Description: Targeted devices enrolled in Intune. **Why:** Certificate profiles apply only to enrolled devices. **Verify:** Intune → **Devices** → **All devices**; run `CPKI-Discovery.ps1`. **Implement:** Enable automatic enrollment; enroll the target devices.

2.3 Cloud PKI Intune connector / plugin connected

Description: The cloud PKI's Intune connector or plugin is registered and connected to your tenant. **Why:** Without the connector, Intune cannot request certificates from the cloud PKI's issuing CA. **Verify:** Intune → **Tenant administration** → **Connectors and tokens** → **Certificate connectors** (and the cloud PKI's portal); run `CPKI-Discovery.ps1` (lists certificate connectors). Confirm the connector shows **Active/healthy** with a recent check-in. **Implement:** Follow the **vendor's documentation** to install/register the connector or plugin and grant it the required Graph permissions; confirm it appears healthy in Intune.

2.4 Trust chain available (root + any intermediate)

Description: The cloud PKI's **root** CA certificate and any **intermediate** CA certificate(s), exported as `.cer` (DER) files. **Why:** A leaf certificate is useless on the device without a trusted chain; the chain must be deployed before issuance. **Verify:** You have the root (and intermediate) `.cer` from the cloud PKI portal; `certutil -dump <file>.cer` shows the expected subject/issuer. **Implement:** Export the root and intermediate CA certificates from the cloud PKI (per vendor docs) as DER `.cer` files for upload to Intune Trusted certificate profiles.

2.5 Issuing CA / certificate template ready in the cloud PKI

Description: An issuing CA (and its certificate template/profile) configured in the cloud PKI for the certificates you will issue (device auth, user auth). **Why:** Intune SCEP/PKCS profiles request from a specific issuing CA/template; it must exist and allow the intended subject/SAN, key usage, and validity. **Verify:** The cloud PKI portal shows the issuing CA and template with the intended EKU (e.g. Client Authentication), key size, and validity; note the **SCEP URL** / connector identifier the vendor provides.

Implement: Create/confirm the issuing CA and template in the cloud PKI (per vendor docs); record the SCEP URL / template name for the Intune profile.

2.6 Certificate scope groups (pilot + broad)

Description: Entra security groups that scope certificate issuance — at minimum a pilot and a broad group, split by device vs user as needed. **Why:** Issuance is piloted first; user vs device certificates assign to different group types. **Verify:** Entra → **Groups**; the scope groups exist; or build them with `CPKI-Set-ScopeTargeting.ps1`. **Implement:** Create `Cloud PKI Cert – Devices – Pilot` / `– Broad` and `Cloud PKI Cert – Users – Pilot` / `– Broad` (use `CPKI-Set-ScopeTargeting.ps1`).

2.7 Certificate-backed scenario targets (Wi-Fi / VPN / auth)

Description: The Wi-Fi/VPN/authentication scenario(s) the certificate will serve, and their existing profiles. **Why:** Validation confirms the *scenario* works, not just that a certificate installed; the certificate profile must match what the scenario expects (EKU, SAN). **Verify:** Intune → **Devices** → **Configuration**; existing Wi-Fi/VPN profiles (or the auth scenario) are identified and reference a certificate. **Implement:** Identify (or create) the Wi-Fi/VPN/auth profile that will consume the issued certificate; note the required certificate properties.

2.8 Legacy CA inventory (for migration)

Description: An inventory of what the existing on-prem CA currently issues (which profiles, which scenarios, which devices) and its connector. **Why:** You cannot safely migrate or retire the legacy CA without knowing what depends on it. **Verify:** Intune → existing SCEP/PKCS and Trusted certificate profiles; the existing certificate connector; run `CPKI-Report.ps1` for current coverage. **Implement:** Document the legacy SCEP/PKCS profiles, the scenarios they back, and the connector — this is the migration checklist.

2.9 Change control / rollback (authentication-impacting)

Description: A change ticket, maintenance window, approver, and documented rollback for the issuance and migration changes. **Why:** Certificate and Wi-Fi/VPN changes can lock users out of the network; a pilot gate and rollback are mandatory (see global G8). **Verify:** The change record exists with an approver and a rollback step. **Implement:** Record the change, name the pilot group, define rollback (unassign new profiles; legacy chain remains trusted), and get approval.

Checklist

- Intune licensing + cloud PKI subscription with Intune integration
- Devices enrolled — `CPKI-Discovery.ps1`
- Cloud PKI Intune connector/plugin connected and healthy — `CPKI-Discovery.ps1`
- Trust chain (root + intermediate) exported as `.cer`
- Issuing CA / template ready in the cloud PKI; SCEP URL / template recorded

- Scope groups created (devices + users, pilot + broad) — `CPKI-Set-ScopeTargeting.ps1`
- Wi-Fi/VPN/auth scenario target identified
- Legacy CA inventory documented — `CPKI-Report.ps1`
- Change control + rollback approved

3. Implementation Guide — Cloud PKI Integration with Intune

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`). The cloud-PKI **product** steps (connector install, issuing CA, certificate template) are done in the **vendor's console per the vendor's documentation** and are flagged below; everything else is the exact Intune-side procedure.

> ***Model:*** *Connect the cloud PKI → deploy the trusted root/intermediate chain → create SCEP/PKCS certificate profiles → issue to a pilot → validate Wi-Fi/VPN/auth → migrate workloads off the legacy CA (coexist, then retire) → expand. Pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand****, with a pilot gate and rollback.*
>
> ***Naming — reuse these exact names throughout*** *(`<Purpose> – <Scope> – <Stage>`): connector* `Cloud PKI Connector – Tenant – Integration`*; trust profiles* `Trusted Root – Cloud PKI – All Devices` *and* `Trusted Intermediate – Cloud PKI – All Devices`*; device cert* `SCEP Cert – Device Auth – Pilot` */ `– Broad`*; user cert* `PKCS Cert – User Auth – Pilot` */ `– Broad`*; scope groups* `Cloud PKI Cert – Devices – Pilot` */ `– Broad` *and* `Cloud PKI Cert – Users – Pilot` */ `– Broad`*.*

Step 1 — Create the scope groups

1. Entra → **Groups** → **New group** → **Security** → `Cloud PKI Cert – Devices – Pilot` (a few representative devices). **Create**.

1. Repeat for `Cloud PKI Cert – Devices – Broad`, `Cloud PKI Cert – Users – Pilot`, and `Cloud PKI Cert – Users – Broad`. (Or build all four with `CPKI-Set-ScopeTargeting.ps1`.)

Step 2 — Connect the cloud PKI to Intune (vendor console)

> ***Vendor step — defer to the cloud PKI vendor's documentation.*** *Install/register the cloud PKI's* ***Intune connector or plugin*** *and grant it the Graph permissions the vendor specifies. Name the connector reference* `Cloud PKI Connector – Tenant – Integration` *where the vendor allows a label.*

1. After the vendor steps, confirm the connector in Intune → **Tenant administration → Connectors and tokens → Certificate connectors**. **It must show** Active/healthy** with a recent check-in.

1. Run `CPKI-Discovery.ps1` to confirm the certificate connector is present and healthy from the Graph side. An unhealthy connector cannot issue certificates — fix it before continuing (see Troubleshooting A).

Step 3 — Deploy the trusted root/intermediate chain (do this before any issuance)

The leaf certificate is useless on the device without a trusted chain, so deploy the chain first.

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.
2. **Platform:** select your platform (e.g. Windows 10 and later / iOS/iPadOS / Android / macOS).

Profile type: Templates → **Trusted certificate**. **Create**.

1. **Name:** `Trusted Root – Cloud PKI – All Devices`. **Next**.
2. **Configuration settings:** upload the cloud PKI **root** CA `.cer` (DER). For Windows, set the

Destination store to **Computer certificate store – Root**. **Next**.

1. **Assignments** → **Add groups** → `Cloud PKI Cert – Devices – Pilot` (and the users pilot group if

you will issue user certificates). **Next** → **Create**.

1. If the cloud PKI has an **intermediate** CA, create a second **Trusted certificate** profile named

`Trusted Intermediate – Cloud PKI – All Devices` and upload the intermediate `.cer` (Windows destination store **Intermediate**). Assign to the same groups.

Repeat Step 3 per platform you target — a Trusted certificate profile is platform-specific.

Step 4 — Create the SCEP device certificate profile (pilot)

Use SCEP for device authentication (the private key is generated on the device).

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.
2. **Platform:** your device platform. **Profile type: Templates** → **SCEP certificate**. **Create**.
3. **Name:** `SCEP Cert – Device Auth – Pilot`. **Next**.
4. **Configuration settings:**

• **Certificate type: Device**.

• **Subject name format:** e.g. `CN={{DeviceName}}` (use the attributes your scenario requires).

• **Subject alternative name:** add the SAN your scenario needs (e.g. DNS or the value the cloud

PKI template expects).

• **Certificate validity period:** match the cloud PKI template (e.g. 1 year).

• **Key storage provider / Key usage / Key size:** per your standard (e.g. 2048).

• **Extended key usage (EKU):** add **Client Authentication** (`1.3.6.1.5.5.7.3.2`) for Wi-Fi/VPN.

• **Renewal threshold (%):** e.g. **20** (auto-renew before expiry).

• **Root certificate:** select the `Trusted Root – Cloud PKI – All Devices` profile from Step 3.

• **SCEP server URLs:** enter the **cloud PKI SCEP URL** from the vendor (Requirement 2.5).

Next.

1. **Assignments** → **Add groups** → `Cloud PKI Cert – Devices – Pilot`. **Next** → **Create**.

*If the cloud PKI uses a challenge/connector mechanism other than a plain SCEP URL, follow the vendor's documentation for the exact value(s) to enter in **SCEP server URLs** — the Intune profile shape above is unchanged.*

Step 5 — (If issuing user certificates) Create the PKCS user certificate profile (pilot)

Use PKCS for user authentication where the cloud PKI's Intune integration supports PKCS issuance.

1. Intune → **Devices** → **Configuration** → **Create** → **New policy**.
2. **Platform:** your platform. **Profile type: Templates** → **PKCS certificate**. **Create**.
3. **Name:** `PKCS Cert – User Auth – Pilot`. **Next**.
4. **Configuration settings:**

• **Certificate type: User**.

• **Subject name format:** e.g. `CN={{UserName}}` ; **SAN: User principal name (UPN)** = `{{UserPrincipalName}}`.

• **Certification authority / CA name / Certificate template name:** enter the **cloud PKI issuing CA and template** identifiers the vendor provides (Requirement 2.5).

• **Validity / Key usage / EKU:** match the template; add **Client Authentication** EKU.

• **Root certificate:** select `Trusted Root – Cloud PKI – All Devices`.

Next.

1. **Assignments** → **Add groups** → `Cloud PKI Cert – Users – Pilot`. **Next** → **Create**.

***PKCS support varies by cloud PKI.** If the vendor's Intune integration does not support PKCS, issue user certificates via SCEP (Certificate type **User**) instead, per the vendor's guidance.*

Step 6 — Point a certificate-backed scenario at the issued certificate (pilot)

Issuing a certificate proves issuance; the scenario must actually *use* it.

1. Intune → **Devices** → **Configuration** → **Create** → **New policy** → your platform → **Templates** → Wi-Fi **(or** VPN**).

1. In the profile's **authentication** settings, select **certificate** authentication and choose the **client certificate** that references your SCEP/PKCS profile (`SCEP Cert – Device Auth – Pilot` or `PKCS Cert – User Auth – Pilot`) and the **root** = `Trusted Root – Cloud PKI – All Devices`.

1. Assign to `Cloud PKI Cert – Devices – Pilot` (Wi-Fi/VPN device auth) or the users pilot group.

Create.

Step 7 — Apply on a pilot device and validate

1. On a pilot device, force a sync (Windows: **Settings → Accounts → Access work or school → Info → Sync**; **iOS/iPadOS:** Settings → General → VPN & Device Management → **management profile** → Sync**), or push** Sync **from Intune (**Devices → (device) → Sync**).

1. Confirm the **trust chain** and the **issued certificate** are present:

• Windows: `certlm.msc` → **Personal** → **Certificates** shows the leaf; **Trusted Root** shows the cloud PKI root.

• Or run `CPKI-Validation.ps1` for the device.

1. Confirm the certificate's **status** in Intune: open `SCEP Cert – Device Auth – Pilot` → **Device status** = **Succeeded**.

1. **Validate the scenario:** connect to the certificate-backed **Wi-Fi/VPN** (or perform the auth action) and confirm it succeeds **using the cloud-issued certificate**.

Step 8 — Pilot gate

Do not expand or migrate until: the connector is healthy, the chain and certificate are present on pilot devices (`CPKI-Validation.ps1`), the profile shows **Succeeded**, **and** the Wi-Fi/VPN/auth scenario works with the cloud-issued certificate. Complete the **Testing Guide**. **Rollback:** if the pilot fails, unassign the new SCEP/PKCS and scenario profiles from the pilot groups — devices remove the certificate at next sync; the legacy CA chain remains trusted, so no production scenario breaks.

Step 9 — Migrate workloads off the legacy CA (coexistence → retire)

Migrate one scenario at a time; both chains stay trusted during coexistence.

1. **Coexist.** Keep the legacy root/intermediate trusted (leave the legacy Trusted certificate profiles assigned) while the cloud PKI chain is also deployed. Devices now trust **both** CAs.

1. **Re-issue.** For each scenario, assign the cloud PKI SCEP/PKCS profile to the broad scope group so devices receive a **new** cloud-issued certificate alongside the legacy one.

1. **Re-point the scenario.** Update each Wi-Fi/VPN/auth profile to use the **cloud PKI** client certificate + root (Step 6) instead of the legacy one. Validate per scenario before moving on.

1. **Drain the legacy CA.** Once a scenario validates on the cloud certificate fleet-wide, unassign the **legacy** SCEP/PKCS profile for that scenario; devices stop renewing from the legacy CA and the legacy certificates age out.

1. **Retire.** When `CPKI-Report.ps1` shows no devices depending on legacy-issued certificates and no scenario references the legacy CA, retire the legacy SCEP/PKCS profiles and (after a safety margin) remove the legacy Trusted certificate profiles and decommission the on-prem CA/connector per change control.

Step 10 — Expand to production

1. Add the **broad** scope groups to the trust, SCEP/PKCS, and scenario profiles (or clone the pilot profiles to `– Broad` and assign the broad groups).
1. Re-validate with `CPKI-Validation.ps1` and run `CPKI-Report.ps1` for fleet coverage + expiry.

Order of operations

`Scope groups → Connect cloud PKI (vendor) → Trusted chain → SCEP device profile → (PKCS user profile) → Point Wi-Fi/VPN/auth at the cert → Apply → Validate → Pilot gate → Migrate off legacy CA (coexist → re-point → drain → retire) → Expand`

5. Testing Guide — Cloud PKI Integration with Intune

Run after the pilot profiles apply (Implementation Step 7) and again after each migration step and at expansion. Use `CPKI-Validation.ps1` for per-device checks and `CPKI-Report.ps1` for fleet coverage.

Test 1 — Connector healthy

Do: Intune → **Tenant administration** → **Connectors and tokens** → **Certificate connectors**; run `CPKI-Discovery.ps1`. **Expected:** The cloud PKI certificate connector shows **Active/healthy** with a recent check-in.

Test 2 — Trust chain installed on the device

Do: On a pilot device, check the root/intermediate (Windows: `certlm.msc` → **Trusted Root** / **Intermediate**), or run `CPKI-Validation.ps1`. **Expected:** The cloud PKI **root** (and intermediate, if any) is present and trusted.

Test 3 — Certificate issued to the device

Do: Intune → `SCEP Cert – Device Auth – Pilot` (and `PKCS Cert – User Auth – Pilot` if used) → **Device status**; or `CPKI-Validation.ps1`. **Expected:** Status **Succeeded**; the leaf certificate is present in the device's **Personal** store, chains to the cloud PKI root, and is within validity.

Test 4 — Certificate properties correct

Do: Inspect the issued certificate (subject/SAN, EKU, validity, issuer) via `CPKI-Validation.ps1` or the certificate store. **Expected:** Subject/SAN match the profile, **issuer = cloud PKI CA**, EKU includes **Client Authentication**, and the validity matches the template.

Test 5 — Wi-Fi scenario works on the cloud certificate

Do: On a pilot device, connect to the certificate-backed Wi-Fi. **Expected:** The device authenticates and connects **using the cloud-issued certificate** (not the legacy one).

Test 6 — VPN / auth scenario works on the cloud certificate

Do: Connect the certificate-backed VPN (or perform the authentication action) on a pilot device. **Expected:** The scenario succeeds using the cloud-issued certificate.

Test 7 — Renewal behaves

Do: Confirm the **renewal threshold** is set on the profile; for a near-expiry test certificate (or by inspection) confirm Intune renews before expiry. **Expected:** Devices auto-renew before expiry; no gap in certificate coverage.

Test 8 — Coexistence (during migration)

Do: During migration, run `CPKI-Validation.ps1`. **Expected:** Both the **legacy** and the **cloud PKI** roots are trusted; the re-pointed scenario uses the cloud certificate; no scenario is broken.

Test 9 — Fleet coverage + expiry report

Do: Run `CPKI-Report.ps1`. **Expected:** A clear per-device certificate coverage report — who has a cloud-issued certificate, issuer, expiry, and any devices still on the legacy CA.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–6 pass on the pilot; the Wi-Fi/VPN/auth scenario works on the cloud-issued certificate
Migration pass (per scenario)	Tests 5/6 pass on the cloud certificate fleet-wide and Test 8 confirms coexistence before draining the legacy CA
Expand pass	Tests 3, 5/6, 9 pass for the broad scope before retiring the legacy CA

6. Troubleshooting Guide — Cloud PKI Integration with Intune

Use `CPKI-Discovery.ps1` for connector/profile state, `CPKI-Validation.ps1` for per-device issuance and trust, `CPKI-Report.ps1` for fleet coverage/expiry, and `CPKI-Remediation.ps1` (default `-WhatIf`) for safe fixes. Cloud-PKI **product** internals (console, connector install, CA template) are deferred to the **vendor's documentation**.

A. Connector unhealthy / no issuance

Symptom	Cause	Remediation
All SCEP/PKCS profiles fail	Cloud PKI Intune connector down or expired	Run `CPKI-Discovery.ps1`; in the vendor console repair/re-register the connector per vendor docs; confirm **Active/healthy** in Intune → Certificate connectors
Connector missing in Intune	Connector never registered or wrong tenant	Re-run the vendor connector setup; confirm it targets this tenant and has the required Graph permissions

B. Trust chain problems

Symptom	Cause	Remediation
Leaf installs but scenario fails	Root/intermediate not deployed or not trusted	Confirm `Trusted Root – Cloud PKI – All Devices` (and intermediate) is assigned to the device's group and shows **Succeeded**; run `CPKI-Validation.ps1`
Chain incomplete	Intermediate missing	Create `Trusted Intermediate – Cloud PKI – All Devices` and assign it to the same groups

C. Certificate not issuing to the device

Symptom	Cause	Remediation
SCEP profile shows Error/Pending	Wrong SCEP URL, challenge failure, or group not assigned	Re-check the **SCEP server URLs** value against vendor docs; confirm the device is in the scope group; `CPKI-Remediation.ps1` (sync + re-evaluate)
PKCS profile fails	CA name / template name wrong, or cloud PKI doesn't support PKCS	Verify the issuing CA + template identifiers with the vendor; if PKCS unsupported, switch user issuance to SCEP (Certificate type **User**)
Subject/SAN rejected	Template doesn't permit the requested subject/SAN	Align the Intune profile's subject/SAN with what the cloud PKI template allows

D. Wi-Fi / VPN / auth scenario fails on the new certificate

- Confirm the scenario profile references the **cloud PKI** client certificate + root, not the legacy

one (Implementation Step 6). Confirm the certificate's **EKU** includes **Client Authentication** and the **SAN** matches what the network/NPS/RADIUS or service expects.

• Check the device actually received the cloud certificate (`CPKI-Validation.ps1`) before blaming the

scenario.

E. Migration / coexistence issues

Symptom	Cause	Remediation
Users lose network mid-migration	Legacy chain removed too early, or scenario re-pointed before issuance completed	Keep **both** roots trusted until devices hold a valid cloud certificate; re-point scenarios only after `CPKI-Report.ps1` confirms coverage; rollback = re-point to legacy
Devices still renewing from legacy CA	Legacy SCEP/PKCS profile still assigned	Drain: unassign the legacy profile per scenario only after the cloud certificate is validated fleet-wide

F. Expiry / renewal

• Near-expiry certificates with no renewal → confirm the **renewal threshold** is set on the SCEP/PKCS

profile and the connector is healthy; `CPKI-Report.ps1` lists upcoming expiries to chase.

G. Diagnostics quick reference

```
# Local device certificate state (Windows)
Get-ChildItem Cert:\LocalMachine\My | Select-Object Subject, Issuer, NotAfter, Thumbprint
Get-ChildItem Cert:\LocalMachine\Root | Where-Object { $_.Subject -like '*<cloud PKI CA name>*' }
# Force an Intune sync
Start-Process -FilePath "C:\Windows\System32\deviceenroller.exe" -ArgumentList "/c /AutoEnrollMDM"
```

Use `CPKI-Discovery.ps1` / `CPKI-Validation.ps1` / `CPKI-Report.ps1` for the Graph-side picture.

H. When to escalate

If the connector is healthy, the trust chain is present and trusted, the device is in the scope group and synced, and the certificate still will not issue (or the scenario still fails on a valid cloud-issued certificate), collect the SCEP/PKCS profile **device status** detail and the cloud PKI connector/issuance logs and open a case with **the cloud PKI vendor** (issuance path) and/or **Microsoft** (Intune profile delivery), as the evidence indicates.

7. Best Practices — Cloud PKI Integration with Intune

• **Chain before leaf, always.** Deploy the trusted root/intermediate (`Trusted Root – Cloud PKI –

All Devices`) *before* assigning any SCEP/PKCS profile — a leaf certificate with no trusted chain is useless and silently fails the scenario.

• **Pilot the issuance, gate the migration.** Issue to `Cloud PKI Cert – Devices – Pilot` first and prove the Wi-Fi/VPN/auth scenario works on the cloud certificate before touching production or the legacy CA. This is the single most important practice — certificate mistakes lock users out.

• **Migrate by coexistence, never by switch-flip.** Keep both the legacy and cloud PKI roots trusted during migration, re-point one scenario at a time, validate, then drain the legacy CA. Retire only when `CPKI-Report.ps1` shows zero legacy dependency.

• **Set a renewal threshold.** Configure auto-renew (e.g. 20%) so certificates renew before expiry — expired client certificates are a self-inflicted Wi-Fi/VPN outage.

• **Match subject/SAN and EKU to the scenario.** Wi-Fi/VPN/auth needs the right SAN and the **Client Authentication** EKU; confirm the cloud PKI template allows exactly what the Intune profile requests.

• **Use SCEP for device, PKCS (or SCEP-user) for user.** SCEP keeps the private key on the device; use PKCS for user certificates where the cloud PKI supports it — confirm PKCS support in vendor docs.

• **Scope with groups, name everything.** Use the `<Purpose> – <Scope> – <Stage>` naming for connector, trust, SCEP/PKCS, and scenario profiles so the integration is auditable and repeatable.

• **Keep the connector healthy.** A down connector stops all issuance; monitor it (`CPKI-Discovery.ps1`) and treat connector health as a production dependency.

• **Defer product internals to the vendor.** The cloud PKI console, connector install, and CA template follow the vendor's documentation; keep your Intune-side procedure consistent regardless of vendor.

• **Report coverage + expiry continuously.** Run `CPKI-Report.ps1` on a cadence to catch laggards, near-expiry certificates, and devices still depending on the legacy CA.

• **Document the cadence and rollback.** Record issuing CA/template, profile settings, scope groups, and the rollback step for audits and repeatability.

8. Deliverables Summary — Cloud PKI Integration with Intune

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What a cloud PKI + Intune integration is, why modernize off the on-prem CA, SCEP vs PKCS, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (cloud-PKI product specifics deferred to vendor)

Document	Purpose
Global Requirements (Security solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: connect cloud PKI, trust chain, SCEP/PKCS profiles, scenario, pilot, migrate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`CPKI-Discovery.ps1`	Reports certificate connector health and existing SCEP/PKCS/trusted-cert profiles; cloud-PKI readiness (read-only)
`CPKI-Validation.ps1`	Per-device certificate issuance + trust-chain validation (read-only)
`CPKI-Report.ps1`	Certificate coverage + expiry across the fleet, incl. legacy-vs-cloud issuer (read-only)
`CPKI-Remediation.ps1`	Safe remediation of failed issuance (sync + re-evaluate); defaults to `-WhatIf`
`CPKI-Set-ScopeTargeting.ps1`	Creates/assigns the certificate-scope Entra groups (devices/users, pilot/broad)

What the buyer receives

- This implementation guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Certificates issued to Intune-managed devices from a cloud PKI — trust chain deployed, Wi-Fi/VPN/auth validated, and the on-prem CA dependency reduced or retired — proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CPKI-Discovery.ps1

Purpose: Enumerates the tenant's Intune certificate connectors (and their health/last check-in) and the existing certificate-related device configuration profiles (Trusted certificate, SCEP, PKCS), so you can confirm the cloud PKI's Intune connector is connected and see what cert infrastructure already exists before you configure issuance. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPKI-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and consent to: DeviceManagementConfiguration.Read.All, DeviceManagementServiceConfig.Read.All. EXPECTED RESULTS A list of certificate connectors with status/last-connection, and a list of trusted-cert / SCEP / PKCS profiles with their assigned group IDs. Flags if no connector is found. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Cloud-PKI PRODUCT-specific issuance APIs are deferred to the vendor's documentation; this script only reads the Intune/Graph side (deviceManagement certificate connectors + deviceConfigurations). #> [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\CPKI-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK and consent to: DeviceManagementConfiguration.Read.All, DeviceManagementServiceConfig.Read.All. EXPECTED RESULTS A list of certificate connectors with status/last-connection, and a list of trusted-cert / SCEP / PKCS profiles with their assigned group IDs. Flags if no connector is found. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. Cloud-PKI PRODUCT-specific issuance APIs are deferred to the vendor's documentation; this script only reads the Intune/Graph side (deviceManagement certificate connectors + deviceConfigurations). #> [CmdletBinding()] param()

CPKI-Remediation.ps1

Purpose: For devices whose cloud PKI SCEP/PKCS profile shows Error/Pending, performs the safe, reversible remediations that resolve the common causes: trigger a device sync/check-in so the certificate profile re-evaluates and re-requests issuance. It does NOT delete certificates, unassign profiles, or touch the cloud PKI CA. By default it only REPORTS what it would do (-WhatIf); pass -Apply to act. HOW TO RUN Report only (default): powershell -ExecutionPolicy Bypass -File .\CPKI-Remediation.ps1 ` -CloudPkiProfileMatch "Cloud PKI" Apply: ... .\CPKI-Remediation.ps1 -CloudPkiProfileMatch "Cloud PKI" -Apply Requires the Microsoft Graph SDK and: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.PrivilegedOperations.All (to send a sync action). EXPECTED RESULTS A list of devices in Error/Pending on the matched cloud PKI cert profiles; with -Apply, a sync action is issued to each so issuance re-evaluates at next check-in. Re-run CPKI-Report.ps1 after a sync cycle to confirm the move to Succeeded. .NOTES White-label — the repeatable way. Safe by default (-WhatIf). Connector repair and cloud-PKI CA/ template fixes are out of scope and deferred to the vendor's documentation (see Troubleshooting A/C). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([string]$CloudPkiProfileMatch = 'Cloud PKI', [switch]$Apply)

How to run: Report only (default): powershell -ExecutionPolicy Bypass -File .\CPKI-Remediation.ps1 ` -CloudPkiProfileMatch "Cloud PKI" Apply: ... .\CPKI-Remediation.ps1 -CloudPkiProfileMatch "Cloud PKI" -Apply Requires the Microsoft Graph SDK and: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.PrivilegedOperations.All (to send a sync action). EXPECTED RESULTS A list of devices in Error/Pending on the matched cloud PKI cert profiles; with -Apply, a sync

action is issued to each so issuance re-evaluates at next check-in. Re-run CPKI-Report.ps1 after a sync cycle to confirm the move to Succeeded. .NOTES White-label — the repeatable way. Safe by default (-WhatIf). Connector repair and cloud-PKI CA/ template fixes are out of scope and deferred to the vendor's documentation (see Troubleshooting A/C). #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([string]$CloudPkiProfileMatch = 'Cloud PKI', [switch]$Apply)

CPKI-Report.ps1

Purpose: Builds a fleet-wide picture of certificate coverage to drive the migration and expansion gates: for each managed device it reports the SCEP/PKCS certificate profile deployment state (Succeeded / Error / Pending) and surfaces which devices are still depending on a legacy CA vs the cloud PKI, plus upcoming expiries to chase. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CPKI-Report.ps1 ` -CloudPkiProfileMatch "Cloud PKI" [-LegacyProfileMatch "Legacy"] [-CsvPath .\cpki-coverage.csv] Requires the Microsoft Graph SDK and: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-profile deployment-state summary (how many devices Succeeded / Error / Pending) for the cloud PKI cert profiles (and legacy, if matched), so coverage and laggards are visible. Optional CSV export. Device-internal certificate expiry dates live on the device — use CPKI-Validation.ps1 per device for exact NotAfter; cloud-PKI PRODUCT issuance APIs are deferred to vendor docs. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CloudPkiProfileMatch = 'Cloud PKI', [string]$LegacyProfileMatch, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\CPKI-Report.ps1 ` -CloudPkiProfileMatch "Cloud PKI" [-LegacyProfileMatch "Legacy"] [-CsvPath .\cpki-coverage.csv] Requires the Microsoft Graph SDK and: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A per-profile deployment-state summary (how many devices Succeeded / Error / Pending) for the cloud PKI cert profiles (and legacy, if matched), so coverage and laggards are visible. Optional CSV export. Device-internal certificate expiry dates live on the device — use CPKI-Validation.ps1 per device for exact NotAfter; cloud-PKI PRODUCT issuance APIs are deferred to vendor docs. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CloudPkiProfileMatch = 'Cloud PKI', [string]$LegacyProfileMatch, [string]$CsvPath)

CPKI-Set-ScopeTargeting.ps1

Purpose: Creates (idempotently) the four Entra security groups used to scope cloud PKI certificate issuance, following the solution naming convention so the Intune trust / SCEP / PKCS / Wi-Fi / VPN profiles can be assigned consistently: Cloud PKI Cert - Devices - Pilot Cloud PKI Cert - Devices - Broad Cloud PKI Cert - Users - Pilot Cloud PKI Cert - Users - Broad Existing groups (matched by displayName) are reused, not duplicated. Members are NOT added — add pilot devices/users by hand or via dynamic rules per your standard. HOW TO RUN Report only (default): powershell -ExecutionPolicy Bypass -File .\CPKI-Set-ScopeTargeting.ps1 Apply: ... .\CPKI-Set-ScopeTargeting.ps1 -Apply Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS The four scope groups exist (created if missing, reused if present), with their object IDs printed for use in the Intune profile assignments. .NOTES White-label — the repeatable way. Safe by default (-WhatIf); pass -Apply to create groups. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([switch]$Apply)

How to run: Report only (default): powershell -ExecutionPolicy Bypass -File .\CPKI-Set-ScopeTargeting.ps1 Apply: ... .\CPKI-Set-ScopeTargeting.ps1 -Apply Requires the Microsoft Graph SDK and Group.ReadWrite.All. EXPECTED RESULTS The four scope groups exist (created if

missing, reused if present), with their object IDs printed for use in the Intune profile assignments. .NOTES White-label — the repeatable way. Safe by default (-WhatIf); pass -Apply to create groups. #> [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='Medium')] param([switch]$Apply)

CPKI-Validation.ps1

Purpose: Run on a pilot/managed Windows device to confirm the cloud PKI integration actually delivered: (1) the cloud PKI root/intermediate is present and trusted, and (2) a leaf client certificate was issued by the cloud PKI CA, chains to that root, has the Client Authentication EKU, and is within validity. This is the device-side proof behind Testing Tests 2-4. Read-only. HOW TO RUN Elevated on the device: powershell -ExecutionPolicy Bypass -File .\CPKI-Validation.ps1 -CloudPkiCaName "" -CloudPkiCaName is matched against certificate Issuer/Subject to identify the cloud PKI chain. EXPECTED RESULTS PASS for: cloud PKI root trusted; a leaf cert issued by the cloud PKI CA present in LocalMachine\My; chain valid; Client Authentication EKU present; not expired. Any FAIL points to the matching Troubleshooting section (B trust chain / C issuance). .NOTES White-label — the repeatable way. Read-only; inspects the local Windows certificate stores only. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$CloudPkiCaName)

How to run: Elevated on the device: powershell -ExecutionPolicy Bypass -File .\CPKI-Validation.ps1 -CloudPkiCaName "<cloud PKI CA name>" -CloudPkiCaName is matched against certificate Issuer/Subject to identify the cloud PKI chain. EXPECTED RESULTS PASS for: cloud PKI root trusted; a leaf cert issued by the cloud PKI CA present in LocalMachine\My; chain valid; Client Authentication EKU present; not expired. Any FAIL points to the matching Troubleshooting section (B trust chain / C issuance). .NOTES White-label — the repeatable way. Read-only; inspects the local Windows certificate stores only. #> [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$CloudPkiCaName)